

AUDITORIA TECNICA COMPLETA

DECODEX Bolivia

Sistema de Monitoreo Legislativo-Institucional

Fecha:	7 de mayo de 2026
Stack:	Next.js 16 + Prisma + SQLite + Bun
Alcance:	7 areas de auditoria, 57 archivos revisados
Metodo:	Revision estatica de codigo + queries DB

RESUMEN EJECUTIVO

Se realizo una auditoria tecnica exhaustiva del sistema DECODEX Bolivia, abarcando 7 areas criticas: schema de base de datos, integridad de datos, calidad de codigo, APIs, prompts de inteligencia artificial, deduplicacion y pipelines, y performance/estabilidad. Se revisaron 57 archivos fuente y se ejecutaron queries de verificacion directa sobre la base de datos SQLite. El sistema se encuentra en fase de desarrollo activo con FASE 4D recién implementada (Intencion del Medio + Ejes por Cliente).

La base de datos esta estructuralmente integra: 0 huérfanos, 0 duplicados, 0 nulos criticos. Sin embargo, se identificaron 41 hallazgos criticos y 76 hallazgos medios distribuidos principalmente en las areas de APIs (falta de autentificacion y validacion), prompts de IA (sin timeouts, inconsistencias en backward-compat), deduplicacion (huellas sin normalizar, perdida silenciosa de menciones), y performance (N+1 queries, ausencia de cache).

Tabla Resumen por Area

Area	Criticos	Medios	Leves	Estado General
A. Prisma Schema	3	15	6	Requiere atencion
B. Integridad DB	0	0	1	OK
C. Codigo	2	9	6	Requiere atencion
D. APIs	19	21	7	Critico
E. Prompts IA	3	6	3	Requiere atencion
F. Deduplicacion	6	7	5	Critico
G. Performance	8	18	6	Critico
TOTAL	41	76	34	

TOP 5 Problemas Mas Urgentes

#	Severidad	Area	Descripcion	Recomendacion
1	CRITICO	Sin timeouts en llamadas LLM (extractor, analyzer, dedup). Si el proveedor de IA se cuelga, el worker se bloquea indefinidamente. Afecta: extractor-mentiones.ts, analyze.ts, deduplicacion.ts		
2	CRITICO	Sin autentificacion en endpoints de escritura. Cualquier request anonimo puede crear, modificar o eliminar datos. Los endpoints /api/admin/*, /api/jobs/worker, /api/jobs/scheduler y /api/seed son los mas urgentes.		

#	Severidad	Area	Descripcion	Recomendacion
3	CRITICO	Deduplicacion: calcular Huella() NO normaliza sus argumentos. La funcion normalizar() existe pero nunca se invoca dentro de calcularHuella(). Esto causa huellas inconsistentes y falsos negativos/positivos en la deduplicacion.		
4	CRITICO	N+1 queries en mantenimiento.ts (500 updates individuales) y extractor-menciones.ts (verificaciones por legislador). Impacta significativamente la latencia por articulo.		
5	CRITICO	8 endpoints exponen mensajes de error internos (stack traces, nombres de modulos) en respuestas HTTP 500. La funcion safeError() existe pero no se usa consistentemente.		

TOP 5 Mejoras de Mayor Impacto

1. Agregar timeouts (30-60s) a las 3 llamadas LLM principales. Este cambio solo previene bloqueos totales del sistema con un esfuerzo minimo.
2. Implementar middleware de autenticacion global para todos los endpoints POST/PUT/DELETE. Protege el sistema completo de accesos no autorizados en un solo cambio.
3. Agregar caché en memoria con TTL (30-60s) para Marco Conceptual, lista de personas y ejes tematicos. Estos datos se cargan desde DB en cada scrape y rara vez cambian.
4. Usar updateMany/createMany en vez de loops con updates/creates individuales. Reduce de 500 queries a 1 en mantenimiento y mejora latencia en extractor.
5. Normalizar correctamente las huellas en calcularHuella() llamando a normalizar() en cada componente antes de concatenar. Mejora la precision de deduplicacion inmediatamente.

Veredicto

El sistema NO esta listo para produccion. Requiere condiciones previas obligatorias antes de desplegar: (1) implementar autenticacion en todos los endpoints de escritura, (2) agregar timeouts a llamadas LLM, (3) corregir la normalizacion de huellas en deduplicacion, (4) agregar rate limiting consistente, y (5) corregir las race conditions en transacciones criticas. El sistema funciona correctamente en modo desarrollo y la base de datos esta integra, pero tiene vulnerabilidades de seguridad y riesgos de estabilidad que deben resolverse antes de exponerlo a usuarios reales o datos de produccion.

AUDITORIA A: PRISMA SCHEMA Y RELACIONES

Estado: ADVERTENCIAS

Se reviso completamente el archivo prisma/schema.prisma que define 26 modelos. Se encontraron 3 problemas criticos relacionados con relaciones faltantes e indices ausentes en campos de alta consulta, 15 problemas medios incluyendo convencion @@map inconsistente y onDelete peligroso, y 6 hallazgos leves.

Hallazgos Criticos y Medios Principales

#	Severidad	Modelo	Descripcion	Recomendacion
1	CRITICO	Mencion	mencionOriginalId sin @relation(). FK autorreferencial sin integridad referencial.	Agregar @relation con onDelete: SetNull
2	CRITICO	Mencion	mediold sin indice. FK mas consultada del sistema.	Agregar @@index([mediold])
3	CRITICO	Mencion	personald sin indice. Consultas por legislador.	Agregar @@index([personald])
4	MEDIO	Mencion.medio	onDelete Cascade por defecto. Eliminar un Medio destruye todas sus Menciones.	Cambiar a onDelete: Restrict
5	MEDIO	Contrato	montoMensual usa Float. Perdida de precision en valores monetarios.	Cambiar a Decimal
6	MEDIO	Global	@@map inconsistente: solo 4 de 21 modelos usan snake_case.	Estandarizar convencion
7	MEDIO	Varios	23+ campos JSON almacenados como String en vez de tipo Json de Prisma.	Cambiar a tipo Json
8	MEDIO	Suscriptor	email sin @unique. Posibles duplicados en envios.	Agregar @unique

Resumen: 3 criticos (FKs sin indice y sin relacion), 15 medios, 6 leves. La prioridad inmediata es agregar indices en mediold, personald y mencionId. La convencion @@map inconsistente no causa errores funcionales pero dificulta mantenimiento a largo plazo. El onDelete Cascade implicito en Medio-Mencion es un riesgo de perdida de datos que debe corregirse antes de produccion.

AUDITORIA B: INTEGRIDAD DE DATOS (DB)

Estado: OK

Se conecto a SQLite (db/custom.db) y ejecutaron queries de verificacion directa. El schema Prisma coincide 100% con el schema SQLite real. Se verificaron 20 relaciones de integridad referencial, 7 checks de duplicados, y 19 checks de campos criticos nulos. Resultado: 0 huérfanos, 0 duplicados, 0 nulos criticos.

Volumetria de Datos

Persona	173	Medio	30	marco_conceptual	1
Mencion	0	Job	0	Cliente	0
Contrato	0	Entrega	0	IndicadorValor	0
EjeTematico	0	Reporte	0	Comentario	0
User	0	Session	0	CapturaLog	0

Tabla 1: Volumetria por tabla. DB en estado temprano de desarrollo.

Resumen: 0 criticos, 0 medios, 1 leve. La base de datos esta estructuralmente integra. PRAGMA integrity_check retorna "ok". PRAGMA foreign_key_check confirma 0 violaciones. El unico hallazgo leve es que PRAGMA foreign_keys esta OFF por defecto en runtime SQLite, aunque Prisma Client deberia activarlo al abrir cada conexion. Verificar este comportamiento en produccion.

AUDITORIA C: CODIGO - ERRORES Y CONSISTENCIA

Estado: ADVERTENCIAS

Se revisaron todos los archivos en src/lib/ (55+ archivos), src/lib/jobs/runners/ (7 archivos), src/lib/services/ (14 archivos), y componentes principales. Se identificaron 2 hallazgos criticos, 9 medios y 6 leves.

Hallazgos Principales

#	Severidad	Archivo:Linea	Descripcion	Recomendacion
1	CRITICO	scheduler.ts:126	Error de DB silenciado completamente al guardar horariosOptimos. catch(() => {}) sin log.	Agregar console.warn en catch
2	CRITICO	deduplicacion.ts:321	any types explicitos para whereClause y candidateResults con eslint-disable.	Usar Prisma.MencionWhereInput
3	MEDIO	extractor-menciones.ts	7 bloques catch {} vacios en crearMencionesExtraidas. Errores al vincular ejes se tragan.	Agregar console.warn en cada catch
4	MEDIO	reportes-utils.ts	9 funciones aceptan menciones: any[] sin tipado. Calculo de sentimiento, top actores, etc.	Crear interfaz MencionReporte
5	MEDIO	fetch-utils.ts:24	addEventListener sin removeEventListener correspondiente. Memory leak potencial.	Agregar removeEventListener en finally
6	MEDIO	Global	Dos sistemas de rate limiting paralelos: rate-limit.ts y proxy.ts con logica diferente.	Consolidar en un solo sistema
7	LEVE	deduplicacion.ts.bak	Archivo .bak residual en codigo fuente. Deberia eliminarse del repositorio.	Eliminar archivo
8	LEVE	indicadores.ts:790	Funcion fetchWithTimeout duplicada localmente. Ya existe en fetch-utils.ts.	Usar version compartida

Resumen: 2 criticos, 9 medios, 6 leves. Los puntos positivos incluyen: todos los setInterval en componentes React tienen cleanup correcto, no se encontraron @ts-ignore, los runners de jobs manejan errores consistentemente con logging y RunnerResult, y la mayoría de rutas API usan safeError() de manera consistente.

AUDITORIA D: APIs - RUTAS, VALIDACIONES Y ERRORES

Estado: ERRORES

Se revisaron las 57 rutas API en src/app/api/. Se identificaron 19 hallazgos criticos, 21 medios y 7 leves. Es el area con mayor cantidad de problemas, principalmente por falta de autenticacion, validacion de entrada inconsistente, y exposicion de errores internos. Nota: la ausencia de autenticacion es esperada en modo desarrollo pero critica para produccion.

Hallazgos Criticos (Top 10)

#	Severidad	Endpoint	Descripcion	Recomendacion
1	CRITICO	admin/purge	POST sin auth. Cualquier usuario puede eliminar todos los clientes, contratos y suscriptores.	Agregar auth middleware admin
2	CRITICO	admin/bulletins/*	6 endpoints POST sin auth. Generacion de boletines con IA (costoso) sin proteccion.	Agregar auth middleware
3	CRITICO	jobs/worker	POST sin auth. Cualquier usuario puede pausar/reanudar el worker del sistema.	Agregar auth admin
4	CRITICO	jobs/scheduler	POST sin auth. Recalculo del scheduler sin proteccion.	Agregar auth admin
5	CRITICO	seed/route.ts	POST con SEED_API_KEY debil. Si env var no configurada, no hay proteccion.	Hacer obligatorio en produccion
6	CRITICO	capture/route.ts	POST sin auth ni rate-limit fuerte. Dispara web searches y clasificacion IA.	Agregar rateGuard + auth
7	CRITICO	analyze/batch	POST expone error.message en respuesta 500. Detalles internos visibles.	Usar safeError()
8	CRITICO	bulletins/* (6)	8 endpoints exponen mensajes de error internos completos en respuestas 500.	Usar safeError()
9	CRITICO	marco-conceptual/versionar	Read-then-write sin transaccion. Race condition puede crear versiones duplicadas.	Envolver en \$transaction()
10	MEDIO	clientes/[id]	PUT sin validacion Zod. Cualquier campo puede inyectarse con tipo arbitrario.	Usar guardedParse()

Aspectos positivos: No se encontro SQL injection (todas las consultas usan Prisma ORM). No hay conflictos de rutas. La funcion safeError() existe y se usa en la mayoría de endpoints. Los metodos HTTP son correctos en su mayoría. El formato de error deberia estandarizarse ({ error: string, code?: string }) para toda la API.

AUDITORIA E: PROMPTS DE IA - CONSISTENCIA Y CALIDAD

Estado: ADVERTENCIAS

Se revisaron los system prompts construidos en extractor-menciones.ts y analyze.ts, incluyendo terminologia, escalas, reglas de exclusion etica, formato JSON de output, y robustez del parsing. Se encontraron 3 criticos, 6 medios y 3 leves.

Hallazgos Principales

#	Severidad	Archivo	Descripcion	Recomendacion
1	CRITICO	extractor, analyze	Sin timeout en llamadas zai.chat.completions.create(). Si el LLM se cuelga, bloquea el worker indefinidamente.	Agregar AbortSignal.timeout(60s)
2	CRITICO	analyze.ts:304	checkProhibitedTerms() nunca detecta nada: espera { terminos: string[] } pero recibe string[] directamente. Funcion completamente inoperativa.	Normalizar formato de entrada
3	CRITICO	extractor:54,510	tratamiento_agregado expuesto al LLM. Valor interno del sistema para deduplicados que el LLM jamas deberia generar.	Remover del prompt y tratamientosValidos
4	MEDIO	analyze.ts	Exclusiones eticas ausentes. El extractor las carga del Marco Conceptual, el analyzer no las tiene.	Agregar al prompt del analyzer
5	MEDIO	analyze.ts	Sin soporte para ejes por cliente (clientId). El extractor lo soporta, el analyzer no.	Evaluar si es necesario por diseno
6	MEDIO	Ambos	Terminologia prohibida: extractor la incluye en prompt (proactivo), analyzer solo verifica post-hoc (reactivo). El LLM del analyzer nunca recibe las reglas.	Incluir reglas en prompt del analyzer
7	LEVE	Ambos	Regex JSON greedy /[{\s\S}]/ puede capturar incorrectamente si hay multiples JSONs.	Usar regex non-greedy o parser robusto

Aspectos positivos: Terminologia "tratamiento periodistico" usada consistentemente. Ambos prompts prohíben la palabra "sentimiento". Las 6 categorias de intencion del medio estan completas e identicas en ambos. Los fallbacks son robustos: todos los campos tienen valores default. El parsing maneja nombres alternativos de campos correctamente.

AUDITORIA F: DEDUPLICACION Y PIPELINES

Estado: ERRORES

Se reviso src/lib/deduplicacion.ts completamente, junto con su integracion en extractor-menciones.ts, capture/route.ts, y los jobs check-fuente y scrape-fuente. Se encontraron 6 criticos, 7 medios y 5 leves. El area de deduplicacion tiene problemas estructurales que afectan la precision y confiabilidad del proceso.

Hallazgos Principales

#	Severidad	Archivo	Descripcion	Recomendacion
1	CRITICO	deduplicacion.ts: 96	calcularHuella() NO normaliza argumentos. La funcion normalizar() existe pero nunca se invoca.	Llamar normalizar() en cada componente
2	CRITICO	extractor-menciones:703	Si deduplicarMencion() falla, la mencion se pierde silenciosamente. No crea original como fallback.	Try/catch propio con fallback a crear original
3	CRITICO	deduplicacion.ts: 321	Busqueda de candidatos sin filtro por tema/eje. Trae TODAS las menciones de otros medios en la ventana.	Agregar filtro por MencionTema
4	CRITICO	deduplicacion.ts: 376	N+1 query: db.medio.findUnique() dentro del loop de candidatos dudosos (hasta 30 queries extra).	Pre-cargar medios en una sola query
5	CRITICO	queue.ts:59	Deque sin bloqueo DB. findFirst + update como dos operaciones sin transaccion ni SELECT FOR UPDATE.	Usar transaccion con row-level locking
6	MEDIO	capture/route.ts: 46	clientId aceptado pero NO pasado a deduplicarMencion() ni analyzeMencion(). Funcionalidad incompleta.	Pasar clientId al extractor
7	MEDIO	deduplicacion.ts: 105	eventold incluye fecha en la huella, fragmentando eventos evolutivos entre dias.	Remover fecha del eventold
8	MEDIO	queue.ts / worker.ts	Jobs en estado en_progreso nunca se recuperan si el worker crasa. No hay stuck job reaper.	Implementar reaper periodico

Resumen: 6 criticos, 7 medios, 5 leves. Los problemas mas urgentes son: (1) la huella no se normaliza correctamente, causando falsos negativos/positivos; (2) las menciones se pierden silenciosamente cuando la deduplicacion falla; (3) la busqueda de candidatos es inefficiente por falta de filtro tematico y N+1 queries; y (4) el sistema de colas no tiene proteccion contra procesamiento duplicado.

AUDITORIA G: PERFORMANCE Y ESTABILIDAD

Estado: ERRORES

Se revisaron los endpoints del dashboard, workers, scheduler, y funciones de datos maestros. Se encontraron 8 criticos, 18 medios y 6 leves. Los problemas principales son: ausencia de timeouts en LLM, queries N+1, datos maestros recargados en cada llamada sin cache, y configuracion de SQLite sin busy_timeout.

Hallazgos Principales

#	Severidad	Archivo	Descripcion	Recomendacion
1	CRITICO	extractor:491	zai.chat.completions.create() sin timeout. LLM colgado = worker bloqueado indefinidamente.	Agregar AbortSignal.timeout(60s)
2	CRITICO	analyze:231	Igual: sin timeout en llamada LLM del analyzer.	Agregar timeout
3	CRITICO	deduplicacion:226	verificarConLLM() sin timeout. Se llama hasta 3 veces por mencion.	Agregar timeout 15-20s
4	CRITICO	mantenimiento:89	500 update() individuales en loop. N+1 query masivo.	Usar updateMany()
5	CRITICO	extractor:386,404	Marco Conceptual, personas y ejes cargados desde DB en CADA llamada a extraerMencionesDeTexto().	Caché en memoria con TTL 60s
6	MEDIO	entregas-hoy:7	new PrismaClient() en vez de singleton compartido. Crea conexiones innecesarias.	Importar de @/lib/db
7	MEDIO	dev.sh	Sin trap para SIGTERM/SIGINT. Proceso hijo puede quedar zombie.	Agregar trap cleanup
8	MEDIO	db.ts	Sin busy_timeout para SQLite. Worker + scheduler + API comparten conexion.	Configurar WAL + busy_timeout
9	MEDIO	Dashboard	entregas-hoy polleado cada 15s. Endpoint con 6+ queries DB.	Aumentar a 30s
10	LEVE	rate-limit.ts	Rate limiting en memoria (Map). No funciona en multi-instancia.	Documentar limitacion

Aspectos positivos: Los intervals del dashboard tienen cleanup correcto con clearInterval. mockJobs tiene purgeCompleted() (aunque no se llama automaticamente). Los fetch intervals son razonables en su mayoría (10-60s). safeError() oculta detalles internos del cliente correctamente. No se observan API keys en logs.

RECOMENDACION DE PROXIMOS PASOS (PRIORIZADOS)

P0 - Inmediato (esta semana)

- Agregar timeouts (30-60s) a las 3 llamadas LLM en extractor, analyzer y dedup
- Implementar auth middleware global para endpoints POST/PUT/DELETE
- Corregir calcularHuella() para que normalice correctamente sus argumentos
- Reemplazar error.message expuesto por safeError() en los 8 endpoints afectados
- Agregar try/catch con fallback en deduplicarMencion() para no perder menciones

P1 - Corto plazo (proximas 2 semanas)

- Agregar indices en medioid, personalid, mencionId en el schema Prisma
- Implementar cache en memoria con TTL para datos maestros (marco, personas, ejes)
- Corregir checkProhibitedTerms() formato de datos en analyze.ts
- Remover tratamiento_agregado del prompt del LLM
- Envolver marco-conceptual/versionar y admin/purge en transacciones Prisma
- Usar updateMany/createMany en vez de loops individuales

P2 - Medio plazo (proximas 4 semanas)

- Estandarizar formato de error en toda la API
- Migrar ejes y suscriptores PUT/DELETE a rutas [id] RESTful
- Agregar filtro por tema en busqueda de candidatos de deduplicacion
- Implementar stuck job reaper para jobs en_progreso
- Agregar exclusiones eticas al prompt del analyzer
- Configurar WAL mode y busy_timeout en SQLite

P3 - Largo plazo

- Estandarizar @@map() en todos los modelos Prisma
- Consolidar los dos sistemas de rate limiting
- Cambiar campos JSON String a tipo Json de Prisma
- Migrar a Prisma Migrations (de db push) para historial de cambios
- Considerar migrar medio.nivel de String a Int